

3

Turing Computability

A function is effectively computable if there are definite, explicit rules by following which one could in principle compute its value for any given arguments. This notion will be further explained below, but even after further explanation it remains an intuitive notion. In this chapter we pursue the analysis of computability by introducing a rigorously defined notion of a Turing-computable function. It will be obvious from the definition that Turing-computable functions are effectively computable. The hypothesis that, conversely, every effectively computable function is Turing computable is known as Turing's thesis. This thesis is not obvious, nor can it be rigorously proved (since the notion of effective computability is an intuitive and not a rigorously defined one), but an enormous amount of evidence has been accumulated for it. A small part of that evidence will be presented in this chapter, with more in chapters to come. We first introduce the notion of Turing machine, give examples, and then present the official definition of what it is for a function to be computable by a Turing machine, or Turing computable.

A superhuman being, like Zeus of the preceding chapter, could perhaps write out the whole table of values of a one-place function on positive integers, by writing each entry twice as fast as the one before; but for a human being, completing an infinite process of this kind is impossible in principle. Fortunately, for human purposes we generally do not need the whole table of values of a function f , but only need the values one at a time, so to speak: given some argument n , we need the value $f(n)$. If it is possible to produce the value $f(n)$ of the function f for argument n whenever such a value is needed, then that is almost as good as having the whole table of values written out in advance.

A function f from positive integers to positive integers is called *effectively computable* if a list of instructions can be given that in principle make it possible to determine the value $f(n)$ for any argument n . (This notion extends in an obvious way to two-place and many-place functions.) The instructions must be completely definite and explicit. They should tell you at each step what to do, not tell you to go ask someone else what to do, or to figure out for yourself what to do: the instructions should require no external sources of information, and should require no ingenuity to execute, so that one might hope to automate the process of applying the rules, and have it performed by some mechanical device.

There remains the fact that for all but a finite number of values of n , it will be infeasible in practice for any human being, or any mechanical device, actually to carry

out the computation: in principle it could be completed in a finite amount of time if we stayed in good health so long, or the machine stayed in working order so long; but in practice we will die, or the machine will collapse, long before the process is complete. (There is also a worry about finding enough space to store the intermediate results of the computation, and even a worry about finding enough matter to use in writing down those results: there's only a finite amount of paper in the world, so you'd have to write smaller and smaller without limit; to get an infinite number of symbols down on paper, eventually you'd be trying to write on molecules, on atoms, on electrons.) But our present study will ignore these practical limitations, and work with an idealized notion of computability that goes beyond what actual people or actual machines can be sure of doing. Our eventual goal will be to prove that certain functions are *not* computable, *even if* practical limitations on time, speed, and amount of material could somehow be overcome, and for this purpose the essential requirement is that our notion of computability not be too *narrow*.

So far we have been sliding over a significant point. When we are given as argument a number n or pair of numbers (m, n) , what we in fact are directly given is a *numeral* for n or an ordered pair of *numerals* for m and n . Likewise, if the value of the function we are trying to compute is a number, what our computations in fact end with is a *numeral* for that number. Now in the course of human history a great many systems of numeration have been developed, from the primitive *monadic* or *tally* notation, in which the number n is represented by a sequence of n strokes, through systems like Roman numerals, in which bunches of five, ten, fifty, one-hundred, and so forth strokes are abbreviated by special symbols, to the *Hindu–Arabic* or *decimal* notation in common use today. Does it make a difference in a definition of computability which of these many systems we adopt?

Certainly computations can be *harder in practice* with some notations than with others. For instance, multiplying numbers given in decimal numerals (expressing the product in the same form) is easier in practice than multiplying numbers given in something like Roman numerals. Suppose we are given two numbers, expressed in Roman numerals, say XXXIX and XLVIII, and are asked to obtain the product, also expressed in Roman numerals. Probably for most of us the easiest way to do this would be first to translate from Roman to Hindu–Arabic—the rules for doing this are, or at least used to be, taught in primary school, and in any case can be looked up in reference works—obtaining 39 and 48. Next one would carry out the multiplication in our own more convenient numeral system, obtaining 1872. Finally, one would translate the result back into the inconvenient system, obtaining MDCCCLXXII. Doing all this is, of course, harder than simply performing a multiplication on numbers given by decimal numerals to begin with.

But the example shows that when a computation can be done in one notation, it is *possible in principle* to do in any other notation, simply by translating the data from the difficult notation into an easier one, performing the operation using the easier notation, and then translating the result back from the easier to the difficult notation. If a function is effectively computable when numbers are represented in one system of numerals, it will also be so when numbers are represented in any other system of numerals, provided only that translation between the systems can itself be

carried out according to explicit rules, which is the case for any historical system of numeration that we have been able to decipher. (To say we have been able to decipher it amounts to saying that there are rules for translating back and forth between it and the system now in common use.) For purposes of framing a rigorously defined notion of computability, it is convenient to use monadic or tally notation.

A *Turing machine* is a specific kind of idealized machine for carrying out computations, especially computations on positive integers represented in monadic notation. We suppose that the computation takes place on a tape, marked into squares, which is unending in both directions—either because it is actually infinite or because there is someone stationed at each end to add extra blank squares as needed. Each square either is *blank*, or has a *stroke* printed on it. (We represent the blank by S_0 or 0 or most often B , and the stroke by S_1 or $|$ or most often 1, depending on the context.) And with at most a finite number of exceptions, all squares are blank, both initially and at each subsequent stage of the computation.

At each stage of the computation, the computer (that is, the human or mechanical agent doing the computation) is *scanning* some one square of the tape. The computer is capable of erasing a stroke in the scanned square if there is one there, or of printing a stroke if the scanned square is blank. And he, she, or it is capable of movement: one square to the right or one square to the left at a time. If you like, think of the machine quite crudely, as a box on wheels which, at any stage of the computation, is over some square of the tape. The tape is like a railroad track; the ties mark the boundaries of the squares; and the machine is like a very short car, capable of moving along the track in either direction, as in Figure 3-1.

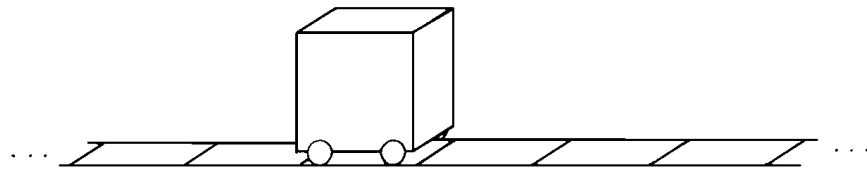


Figure 3-1. A Turing machine.

At the bottom of the car there is a device that can read what's written between the ties, and erase or print a stroke. The machine is designed in such a way that at each stage of the computation it is in one of a finite number of internal *states*, $q_1, \dots, q_m$. Being in one state or another might be a matter of having one or another cog of a certain gear uppermost, or of having the voltage at a certain terminal inside the machine at one or another of m different levels, or what have you: we are not concerned with the mechanics or the electronics of the matter. Perhaps the simplest way to picture the thing is quite crudely: inside the box there is a little man, who does all the reading and writing and erasing and moving. (The box has no bottom: the poor mug just walks along between the ties, pulling the box along.) This operator inside the machine has a list of m instructions written down on a piece of paper and *is in state q_i when carrying out instruction number i* .

Each of the instructions has conditional form: it tells what to do, depending on whether the symbol being scanned (the symbol in the scanned square) is the blank or

stroke, S_0 or S_1 . Namely, there are five things that can be done:

- (1) Erase: write S_0 in place of whatever is in the scanned square.
- (2) Print: write S_1 in place of whatever is in the scanned square.
- (3) Move one square to the right.
- (4) Move one square to the left.
- (5) Halt the computation.

[In case the square is already blank, (1) amounts to doing nothing; in case the square already has a stroke in it, (2) amounts to doing nothing.] So depending on what instruction is being carried out (= what state the machine, or its operator, is in) and on what symbol is being scanned, the machine or its operator will perform one or another of these five overt acts. Unless the computation has halted (overt act number 5), the machine or its operator will perform also a covert act, in the privacy of box, namely, the act of determining what the *next* instruction (*next* state) is to be. Thus the *present* state and the presently *scanned* symbol determine what overt *act* is to be performed, and what the *next state* is to be.

The overall *program* of instructions can be specified in various ways, for example, by a *machine table*, or by a *flow chart* (also called a *flow graph*), or by a *set of quadruples*. For the case of a machine that writes three symbols S_1 on a blank tape and then halts, scanning the leftmost of the three, the three sorts of description are illustrated in Figure 3-2.

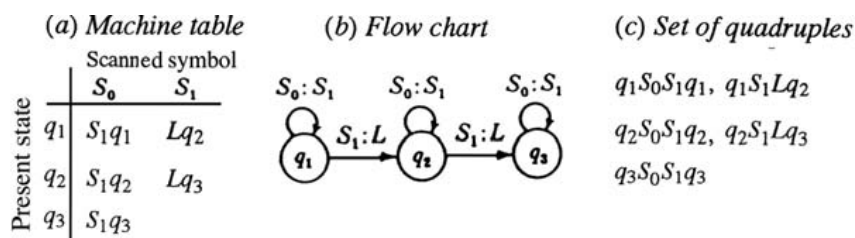


Figure 3-2. A Turing machine program.

3.1 Example (Writing a specified number of strokes). We indicate in Figure 3-2 a machine that will write the symbol S_1 three times. A similar construction works for any specified symbol and any specified number of times. The machine will write an S_1 on the square it's initially scanning, move left one square, write an S_1 there, move left one more square, write an S_1 there, and halt. (It halts when it has no further instructions.) There will be three states—one for each of the symbols S_1 that are to be written. In Figure 3-2, the entries in the top row of the machine table (under the horizontal line) tell the machine or its operator, when following instruction q_1 , that (1) an S_1 is to be written and instruction q_1 is to be repeated, if the scanned symbol is S_0 , but that (2) the machine is to move left and follow instruction q_2 next, if the scanned symbol is S_1 . The same information is given in the flow chart by the two arrows that emerge from the node marked q_1 ; and the same information is also given by the first two quadruples. The significance

in general of a table entry, of an arrow in a flow chart, and of a quadruple is shown in Figure 3-3.

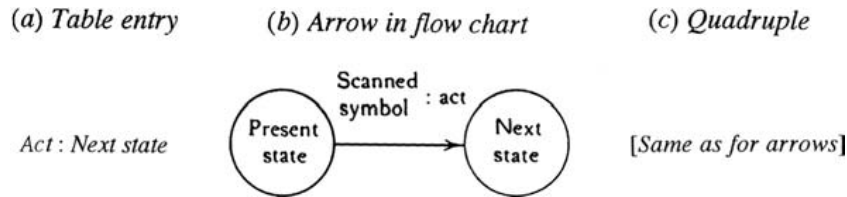


Figure 3-3. A Turing machine instruction.

Unless otherwise stated, it is to be understood that a machine starts in its lowest-numbered state. The machine we have been considering *halts* when it is in state q_3 scanning S_1 , for there is no table entry or arrow or quadruple telling it what to do in such a case. A virtue of the flow chart as a way of representing the machine program is that if the starting state is indicated somehow (for example, if it is understood that the leftmost node represents the starting state unless there is an indication to the contrary), then we can dispense with the names of the states: It doesn't matter what you call them. Then the flow chart could be redrawn as in Figure 3-4.

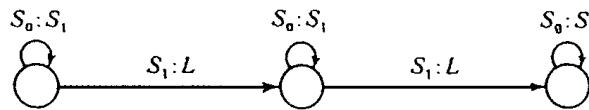


Figure 3-4. Writing three strokes.

We can indicate how such a Turing machine operates by writing down its sequence of *configurations*. There is one configuration for each stage of the computation, showing what's on the tape at that stage, what state the machine is in at that stage, and which square is being scanned. We can show this by writing out what's on the tape and writing the name of the present state under the symbol in the scanned square; for instance,

$$\begin{array}{c} 1100111 \\ 2 \end{array}$$

shows a string or block of two strokes followed by two blanks followed by a string or block of three strokes, with the machine scanning the leftmost stroke and in state 2. Here we have written the symbols S_0 and S_1 simply as 0 and 1, and similarly the state q_2 simply as 2, to save needless fuss. A slightly more compact representation writes the state number as a subscript on the symbol scanned: $1_2100111$.

This same configuration could be written $01_2100111$ or $1_21001110$ or $01_21001110$ or $001_2100111$ or $\dots$ —a block of 0s can be written at the beginning or end of the tape, and can be shorted or lengthened *ad lib.* without changing the significance: the tape is understood to have as many blanks as you please at each end.

We can begin to get a sense of the power of Turing machines by considering some more complex examples.

3.2 Example (Doubling the number of strokes). The machine starts off scanning the leftmost of a block of strokes on an otherwise blank tape, and winds up scanning the leftmost of a block of twice that many strokes on an otherwise blank tape. The flow chart is shown in Figure 3-5.

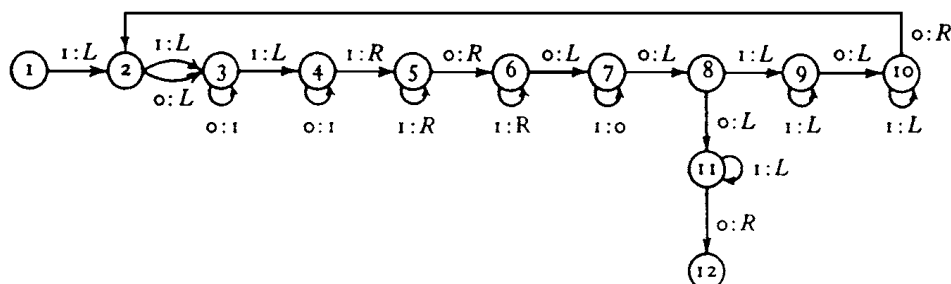


Figure 3-5. Doubling the number of strokes.

How does it work? In general, by writing double strokes at the left and erasing single strokes at the right. In particular, suppose the initial configuration is $1_1 11$, so that we start in state 1, scanning the leftmost of a block of three strokes on an otherwise blank tape. The next few configurations are as follows:

$$0_2 111 \quad 0_3 0111 \quad 1_3 0111 \quad 0_4 10111 \quad 1_4 10111.$$

So we have written our first double stroke at the left—separated from the original block 111 by a blank. Next we go right, past the blank to the right-hand end of the original block, and erase the rightmost stroke. Here is how that works, in two phases. Phase 1:

$$11_5 0111 \quad 11_5 111 \quad 11_6 0111 \quad 11_6 1011 \quad 11_6 1011 \quad 11_6 10110.$$

Now we know that we have passed the last of the original block of strokes, so (phase 2) we back up, erase one of them, and move one more square left:

$$11_7 10111 \quad 11_7 10110 \quad 11_8 10110.$$

Now we hop back left, over what is left of the original block of strokes, over the blank separating the original block from the additional strokes we have printed, and over those additional strokes, until we find the blank beyond the leftmost stroke:

$$11_9 1011 \quad 11_9 1011 \quad 1_{10} 011 \quad 1_{10} 1011 \quad 0_{10} 11011.$$

Now we will print another two new strokes, much as before:

$$0_{12} 1011 \quad 0_3 11011 \quad 1_3 11011 \quad 0_4 111011 \quad 1_4 111011.$$

We are now back on the leftmost of the block of newly printed strokes, and the process that led to finding and erasing the rightmost stroke will be repeated, until we arrive at the following:

$$111_7 1011 \quad 111_7 1010 \quad 111_8 1010.$$

Another round of this will lead first to writing another pair of strokes:

$$1_4 1111101.$$

It will then lead to erasing the last of the original block of strokes:

$$11111101_7 \quad 11111100_7 \quad 1111110_8 0.$$

And now the endgame begins, for we have what we want on the tape, and need only move back to halt on the leftmost stroke:

$$\begin{array}{cccccc} 111111_{11} & 11111_{11}1 & 1111_{11}11 & 111_{11}111 & 11_{11}1111 & 1_{11}11111 \\ 0_{11}111111 & 1_{12}11111. & & & & \end{array}$$

Now we are in state 12, scanning a stroke. Since there is no arrow from that node telling us what to do in such a case, we halt. The machine performs as advertised.

(*Note:* The fact that the machine doubles the number of strokes when the original number is three is not a proof that the machine performs as advertised. But our examination of the special case in which there are three strokes initially made no essential use of the fact that the initial number was three: it is readily converted into a proof that the machine doubles the number of strokes no matter how long the original block may be.)

Readers may wish, in the remaining examples, to try to design their own machines before reading our designs; and for this reason we give the statements of all the examples first, and collect all the proofs afterward.

3.3 Example (Determining the parity of the length of a block of strokes). There is a Turing machine that, started scanning the leftmost of an unbroken block of strokes on an otherwise blank tape, eventually halts, scanning a square on an otherwise blank tape, where the square contains a blank or a stroke depending on whether there were an even or an odd number of strokes in the original block.

3.4 Example (Adding in monadic (tally) notation). There is a Turing machine that does the following. Initially, the tape is blank except for two solid blocks of strokes, say a left block of p strokes and a right block of q strokes, separated by a single blank. Started on the leftmost blank of the left block, the machine eventually halts, scanning the leftmost stroke in a solid block of $p + q$ strokes on an otherwise blank tape.

3.5 Example (Multiplying in monadic (tally) notation). There is a Turing machine that does the same thing as the one in the preceding example, but with $p \cdot q$ in place of $p + q$.

Proofs

Example 3.3. A flow chart for such a machine is shown in Figure 3-6.

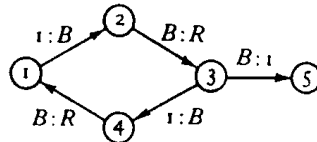


Figure 3-6. Parity machine.

If there were 0 or 2 or 4 or ... strokes to begin with, this machine halts in state 1, scanning a blank on a blank tape; if there were 1 or 3 or 5 or ..., it halts in state 5, scanning a stroke on an otherwise blank tape.

But if there are any strokes left in the counter, the machine goes into a *leapfrog routine*: in effect, it moves the block of q strokes (the *leapfrog group*) q places to the right along the tape. For example, with $p = 2$ and $q = 3$ the tape looks like this initially:

11B111

and looks like this after going through the leapfrog routine:

B1BBBB111.

The machine will then note that there is only one 1 left in the counter, and will finish up by erasing that 1, moving right two squares, and changing all B s to strokes until it comes to a stroke, at which point it continues to the leftmost 1 and halts.

The general picture of how the leapfrog routine works is shown in Figure 3-8.

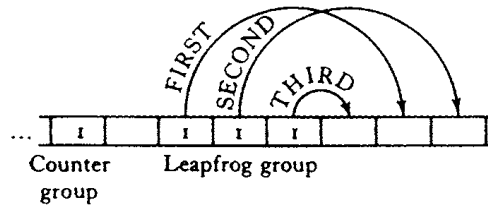


Figure 3-8. Leapfrog.

In general, the leapfrog group consists of a block of 0 or 1 or ... or q strokes, followed by a blank, followed by the remainder of the q strokes. The blank is there to tell the machine when the leapfrog game is over: without it the group of q strokes would keep moving right along the tape forever. (In playing leapfrog, the portion of the q strokes to the left of the blank in the leapfrog group functions as a counter: it controls the process of adding strokes to the portion of the leapfrog group to the right of the blank. That is why there are two big loops in the flow chart: one for each counter-controlled subroutine.)

We have not yet given an official definition of what it is for a numerical function to be computable by a Turing machine, specifying how inputs or arguments are to be represented on the machine, and how outputs or values represented. Our specifications for a k -place function from positive integers to positive integers are as follows:

- (a) The arguments $m_1, \dots, m_k$ of the function will be represented in monadic notation by blocks of those numbers of strokes, each block separated from the next by a single blank, on an otherwise blank tape. Thus, at the beginning of the computation of, say, $3 + 2$, the tape will look like this: 111B11.
- (b) Initially, the machine will be scanning the leftmost 1 on the tape, and will be in its initial state, state 1. Thus in the computation of $3 + 2$, the initial configuration will be 1_111B11 . A configuration as described by (a) and (b) is called a *standard initial configuration* (or *position*).
- (c) If the function that is to be computed assigns a value n to the arguments that are represented initially on the tape, then the machine will eventually halt on a tape

containing a block of that number of strokes, and otherwise blank. Thus in the computation of $3 + 2$, the tape will look like this: 11111.

- (d) In this case, the machine will halt scanning the leftmost 1 on the tape. Thus in the computation of $3 + 2$, the final configuration will be $1_n 1111$, where n th state is one for which there is no instruction what to do if scanning a stroke, so that in this configuration the machine will be halted. A configuration as described by (c) and (d) is called a *standard final configuration* (or *position*).
- (e) If the function that is to be computed assigns no value to the arguments that are represented initially on the tape, then the machine either will never halt, or will halt in some nonstandard configuration such as $B_n 11111$ or $B 11_n 111$ or $B 11111_n$.

The restriction above to the standard position (scanning the leftmost 1) for starting and halting is inessential, but *some* specifications or other have to be made about initial and final positions of the machine, and the above assumptions seem especially simple.

With these specifications, any Turing machine can be seen to compute a function of one argument, a function of two arguments, and, in general, a function of k arguments for each positive integer k . Thus consider the machine specified by the single quadruple $q_1 1 1 q_2$. Started in a standard initial configuration, it immediately halts, leaving the tape unaltered. If there was only a single block of strokes on the tape initially, its final configuration will be standard, and thus this machine computes the identity function id of one argument: $\text{id}(m) = m$ for each positive integer m . Thus the machine computes a certain total function of one argument. But if there were two or more blocks of strokes on the tape initially, the final configuration will not be standard. Accordingly, the machine computes the extreme partial function of two arguments that is undefined for all pairs of arguments: the empty function e_2 of two arguments. And in general, for k arguments, this machine computes the empty function e_k of k arguments.

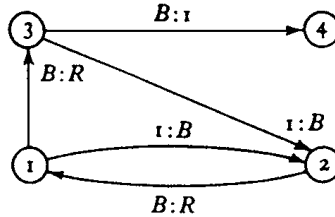


Figure 3-9. A machine computing the value 1 for all arguments.

By contrast, consider the machine whose flow chart is shown in Figure 3-9. This machine computes for each k the total function that assigns the same value, namely 1, to each k -tuple. Started in initial state 1 in a standard initial configuration, this machine erases the first block of strokes (cycling between states 1 and 2 to do so) and goes to state 3, scanning the second square to the right of the first block. If it sees a blank there, it knows it has erased the whole tape, and so prints a single 1 and halts in state 4, in a standard configuration. If it sees a stroke there, it re-enters the cycle between states 1 and 2, erasing the second block of strokes and inquiring again, in state 3, whether the whole tape is blank, or whether there are still more blocks to be dealt with.

A numerical function of k arguments is *Turing computable* if there is some Turing machine that computes it in the sense we have just been specifying. Now computation in the Turing-machine sense is certainly *one* kind of computation in the intuitive sense, so all Turing-computable functions are effectively computable. *Turing's thesis* is that, conversely, any effectively computable function is Turing computable, so that computation in the precise technical sense we have been developing coincides with effective computability in the intuitive sense.

It is easy to imagine liberalizations of the notion of the Turing machine. One could allow machines using more symbols than just the blank and the stroke. One could allow machines operating on a rectangular grid, able to move up or down a square as well as left or right. Turing's thesis implies that no liberalization of the notion of Turing machine will enlarge the class of functions computable, because all functions that are effectively computable in any way at all are already computable by a Turing machine of the restricted kind we have been considering. Turing's thesis is thus a bold claim.

It *is* possible to give a heuristic argument for it. After all, effective computation consists of moving around and writing and perhaps erasing symbols, according to definite, explicit rules; and surely writing and erasing symbols can be done stroke by stroke, and moving from one place to another can be done step by step. But the main argument will be the accumulation of examples of effectively computable functions that we succeed in showing are Turing computable. So far, however, we have had just a few examples of Turing machines computing numerical functions, that is, of effectively computable functions that we have proved to be Turing computable: addition and multiplication in the preceding section, and just now the identity function, the empty function, and the function with constant value 1.

Now addition and multiplication are just the first two of a series of arithmetic operations all of which are effectively computable. The next item in the series is exponentiation. Just as multiplication is repeated addition, so exponentiation is repeated multiplication. (Then repeated exponentiation gives a kind of super-exponentiation, and so on. We will investigate this general process of defining new functions from old in a later chapter.) If Turing's thesis is correct, there must be a Turing machine for each of these functions, computing it. Designing a multiplier was already difficult enough to suggest that designing an exponentiator would be quite a challenge, and in any case, the direct approach of designing a machine for each operation would take us forever, since there are infinitely many operations in the series. Moreover, there are many other effectively computable numerical functions besides the ones in this series. When we return, in the chapter after next, to the task of showing various effectively computable numerical functions to be Turing computable, and thus accumulating evidence for Turing's thesis, a less direct approach will be adopted, and all the operations in the series that begins with addition and multiplication will be shown to be Turing computable in one go.

For the moment, we set aside the positive task of showing functions to be Turing computable and instead turn to examples of numerical functions of one argument that are Turing *un*computable (and so, if Turing's thesis is correct, effectively uncomputable).

Problems

- 3.1** Consider a tape containing a block of n strokes, followed by a space, followed by a block of m strokes, followed by a space, followed by a block of k strokes, and otherwise blank. Design a Turing machine that when started on the leftmost stroke will eventually halt, having neither printed nor erased anything . . .
- (a) . . . on the leftmost stroke of the second block.
 - (b) . . . on the leftmost stroke of the third block.
- 3.2** Continuing the preceding problem, design a Turing machine that when started on the leftmost stroke will eventually halt, having neither printed nor erased anything . . .
- (a) . . . on the rightmost stroke of the second block.
 - (b) . . . on the rightmost stroke of the third block.
- 3.3** Design a Turing machine that, starting with the tape as in the preceding problems, will eventually halt on the leftmost stroke on the tape, which is now to contain a block of n strokes, followed by a blank, followed by a block of $m + 1$ strokes, followed by a blank, followed by a block of k strokes.
- 3.4** Design a Turing machine that, starting with the tape as in the preceding problems, will eventually halt on the leftmost stroke on the tape, which is now to contain a block of n strokes, followed by a blank, followed by a block of $m - 1$ strokes, followed by a blank, followed by a block of k strokes.
- 3.5** Design a Turing machine to compute the function $\min(x, y) =$ the smaller of x and y .
- 3.6** Design a Turing machine to compute the function $\max(x, y) =$ the larger of x and y .